

ScanMeow

Application Documentation

Mobile Scanner · Desktop Client · Cloud Sync
Architecture, Modules & Communication

Version 1.0 — April 2026
Sabina Comandasu & Andra-Maria Smarandache

1. Application Overview

ScanMeow is a cross-platform document scanning system consisting of three interconnected components: an Android mobile application, a Windows desktop client, and a cloud backend built on Supabase. The system allows users to photograph or scan paper documents using their phone camera, process them into PDF files, and have those PDFs appear automatically on their desktop PC — either through a direct Bluetooth transfer or through cloud synchronisation.

Authentication is handled exclusively through Google OAuth, providing a seamless sign-in experience without managing separate credentials. All document storage is tied to the authenticated user's identity, enforced by Supabase Row-Level Security policies so that users can only access their own files.

1.1 High-Level Architecture

Component	Technology	Role
Mobile App	Android (Kotlin + Jetpack Compose)	Camera capture, PDF generation, upload & Bluetooth send
Desktop Client	Python 3 + PyQt5	PDF inbox, Supabase cloud sync, Bluetooth receiver, list management
Cloud Backend	Supabase (PostgreSQL + Storage)	User auth, document metadata table, PDF storage bucket
Vision Prototype	FastAPI + OpenCV (Python)	Experimental edge-detection / de-skew REST endpoint

1.2 Data Flow Summary

- **Scan** → User taps Scan in the mobile app; CameraX captures the image.
- **Process** → The image is converted to a single-page PDF in memory on the device.
- **Upload** → The PDF bytes are uploaded to Supabase Storage (bucket *scans*) and a row is inserted in the *documents* table.
- **Sync** → The desktop client polls Supabase every 12 seconds, detects the new row, downloads the PDF, and adds it to the local inbox.
- **Bluetooth (fallback)** → After the cloud upload the mobile app also attempts a direct RFCOMM Bluetooth transfer using the SMK2 binary protocol; this path is non-blocking and fails gracefully.
- **Manage** → The user can rename, delete, or open PDFs from both the mobile Cloud screen and the desktop list.

2. Mobile Application (Android)

The mobile app is written entirely in Kotlin and uses Jetpack Compose for its declarative UI. The minimum SDK is API 26 (Android 8.0) and the target is API 35.

2.1 Module Map

Package / File	Responsibility
MainActivity.kt	Single-activity host; navigation state machine; Compose entry point
ui/home/MainHomeScreen.kt	Bottom-nav scaffold with Home, Scan, Cloud, and Profile tabs
ui/screens/	Individual full-screen Composables (Login, Home, Scan, Cloud, Profile, PDF Preview)
ui/components/	Reusable Composables (TopBar, BottomBar, DocumentCard, etc.)
ui/theme/Color.kt	Brand colour palette (BRAND_BLUE, SURFACE_VARIANT, etc.)
viewmodel/	Jetpack ViewModels managing UI state and coroutine scopes
data/model/	Plain Kotlin data classes (DocumentMeta, UserProfile, ...)
data/db/	Local Room database (if used) or Supabase REST wrapper
navigation/	NavHost graph definition, route constants

2.2 Navigation & Screen States

Navigation is implemented as a sealed class **AppScreen** held in a **remember { mutableStateOf() }** variable inside MainActivity. Transitions between screens are driven by lambda callbacks passed down the Composable tree rather than a NavController, keeping the navigation logic visible in one place.

- **AppScreen.Login** — Google Sign-In button; triggers OAuth intent via `ActivityResultLauncher`.
- **AppScreen.Home** — Landing screen after auth; shows recent documents.
- **AppScreen.Scan** — Camera preview (`CameraX ImageCapture`); tap-to-capture.
- **AppScreen.PdfPreview** — Shows the captured image as a PDF preview; "Send to PC" button.
- **AppScreen.Cloud** — Lists documents stored in Supabase; supports rename and delete.
- **AppScreen.Profile** — Displays signed-in Google account info; sign-out.

2.3 Camera & PDF Pipeline

Camera capture uses the **CameraX** Jetpack library (*androidx.camera.**). When the user taps the capture button, an **ImageCapture.takePicture()** call writes a temporary JPEG to the app's cache directory. The JPEG is then loaded as an Android **Bitmap**, encoded into a **PdfDocument** with a single page sized to the image dimensions, and serialised to a **ByteArray** held in memory — no

intermediate file on disk.

2.4 Supabase Integration (Mobile)

The mobile app uses the official **Supabase-kt** SDK (*io.github.jan-tennert.supabase*). Three plugins are configured at application start:

- **GoTrue** — authentication; exchanges the Google ID token for a Supabase session JWT.
- **Storage** — uploads PDF bytes to the *scans* bucket under the path `{userId}/{timestamp}_{filename}`.
- **Postgrest** — inserts a row in *public.documents* with *user_id*, *file_name*, *storage_path*, and *created_at_millis*.

All network calls are dispatched on **Dispatchers.IO** inside a **CoroutineScope** tied to the composables's lifecycle. Upload errors are surfaced as **Toast** messages; failures do not prevent the subsequent Bluetooth attempt.

2.5 Bluetooth Send (SMK2 Protocol)

After a successful (or failed) cloud upload the mobile app attempts to push the same PDF bytes directly to the desktop PC over Bluetooth using the proprietary **SMK2** binary protocol on RFCOMM channel 6. The transfer is fire-and-forget: a failure shows a toast saying the file will sync through the cloud instead.

- The target device MAC address is stored in SharedPreferences and resolved to a BluetoothDevice.
- A BluetoothSocket is created with `createRfcommSocketToServiceRecord(UUID)` on channel 6.
- The SMK2 frame contains a 4-byte magic header, a 4-byte payload length (big-endian), and the raw PDF bytes.
- The send runs in a background coroutine; the UI thread is never blocked.

3. Desktop Application (Windows)

The desktop client is a Python 3 application built with **PyQt5**. It runs on Windows and provides a GUI for receiving, viewing, and managing scanned PDFs. It communicates with Supabase over HTTPS and can also receive files directly from the mobile app via Bluetooth.

3.1 Entry Point

The application is launched with `pythonw main.py` (no console window). The entry point creates a **QApplication**, instantiates **MainWindow**, and starts the Qt event loop.

3.2 Module Map

File	Responsibility
main.py	Entry point; QApplication; MainWindow instantiation
app/main_window.py	Central QMainWindow; menu bar; document list; PDF viewer pane; status bar; thread orchestration
app/supabase_client.py	Pure-Python REST client for Supabase (documents table + scans storage bucket)
app/supabase_sync.py	QThread that polls Supabase every 12 s and emits signals for new PDFs
app/document_manager.py	In-memory + JSON-backed registry of local and cloud documents; rename/delete logic
app/bluetooth_receiver.py	QThread listening on RFCOMM channel 6 for incoming SMK2 frames
app/pdf_viewer.py	Embedded PDF page renderer using fitz (PyMuPDF) + QLabel
app/settings.py / config.py	Persistent QSettings or JSON config for Supabase URL, anon key, inbox path

3.3 MainWindow

MainWindow is the hub of the desktop application. On startup it:

- Restores the last Supabase session from persistent storage (access token, user ID).
- Starts a **SupabaseUserSyncThread** if a valid token is found.
- Starts a **BluetoothPdfReceiver** thread for direct Bluetooth reception.
- Populates the document list from the local inbox directory and cloud metadata.
- Connects all QThread signals to Qt slots for UI updates on the main thread.

Key slots: `_on_pdf_received` adds a new entry to the list; `_on_firebase_status` updates the status bar label; `_on_names_updated` refreshes display names after a cloud sync.

3.4 SupabaseClient

A stateless HTTP client that wraps the Supabase PostgREST and Storage APIs using the **requests** library. All methods accept an **access_token** parameter and use it as a Bearer token.

Method	HTTP call	Purpose
--------	-----------	---------

<code>list_user_documents()</code>	GET <code>/rest/v1/documents?user_id=eq.{uid}</code>	Fetch all document rows for the signed-in user
<code>download_pdf()</code>	GET <code>/storage/v1/object/authenticated/scans/{path}</code>	Download raw PDF bytes from private bucket
<code>upload_pdf_and_row()</code>	POST <code>/storage/v1/object/scans/{path}</code> + POST <code>/rest/v1/documents</code>	Upload a PDF and create the metadata row
<code>delete_document()</code>	DELETE <code>/rest/v1/documents?id=eq.{id}</code> + DELETE <code>/storage/v1/object/scans/{path}</code>	Remove a document and its object
<code>patch_document_file_name()</code>	PATCH <code>/rest/v1/documents?id=eq.{id}</code>	Rename a document
<code>find_recent_document_by_file_name()</code>	GET <code>/rest/v1/documents?file_name=eq.{name}</code>	Double application check before inserting after Bluetooth

The JWT **sub** claim (Supabase user UUID) is extracted locally from the token via the **jwt_sub()** helper without a network round-trip, allowing the `user_id` filter to be applied to every query.

3.5 SupabaseUserSyncThread

A **QThread** subclass that runs an infinite polling loop:

- Calls **list_user_documents()** every 12 seconds.
- Detects token expiry by decoding the JWT *exp* claim locally; pauses polling and emits a status message if expired.
- Compares returned doc IDs against an in-memory set of already-processed IDs.
- Downloads new PDFs via **download_pdf()** and saves them to the inbox directory.
- Emits **pdf_received(path, doc_id, storage_path, display_name)** for each new file.
- Emits **names_updated()** when remote file names differ from local cache.
- Emits **status(str)** for status-bar messages.

3.6 BluetoothPdfReceiver

A **QThread** that opens a Bluetooth server socket on RFCOMM channel 6 and waits for incoming SMK2 connections from the mobile app. On Windows, **socket.AF_BLUETOOTH** with **BTPROTO_RFCOMM** is used directly (PyBluez is not required). The thread is secondary to the Supabase sync path — Bluetooth reception failures are non-fatal.

3.7 DocumentManager

Maintains a unified view of documents combining local files and cloud metadata. Responsibilities:

- Track which cloud doc IDs have already been downloaded (**has_cloud_doc()**).
- Sync display names from Supabase pairs (**sync_cloud_file_names_from_pairs()**).
- Persist the mapping to a JSON sidecar file so it survives application restarts.
- Expose the list for the UI's document list widget.

4. Cloud Backend (Supabase)

The cloud layer is provided entirely by **Supabase**, a hosted open-source Backend-as-a-Service built on PostgreSQL. No custom server-side code is deployed; both clients talk directly to Supabase REST APIs.

4.1 Authentication

Supabase **GoTrue** is configured with Google as the OAuth provider. The mobile app passes the Google ID token to GoTrue, which issues a Supabase JWT (access token + refresh token). The desktop app captures the JWT from a browser-based OAuth redirect.

The access token is a standard JWT. The *sub* claim is the Supabase user UUID and is used as *user_id* in all document rows.

4.2 Database Schema

```
CREATE TABLE public.documents (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  file_name   TEXT NOT NULL,  
  storage_path TEXT NOT NULL,  
  created_at_millis BIGINT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

Row-Level Security policies on public.documents:

Policy	Operation	Condition
select_own_documents	SELECT	auth.uid() = user_id
insert_own_documents	INSERT	auth.uid() = user_id
update_own_documents	UPDATE	auth.uid() = user_id
delete_own_documents	DELETE	auth.uid() = user_id

4.3 Storage Bucket

A private Storage bucket named **scans** holds all PDF files. Objects are stored under the path pattern {user_id}/{timestamp_ms}_{filename}. Storage policies mirror the RLS table policies, restricting each user to their own prefix. Downloads use the /storage/v1/object/authenticated/ endpoint, which validates the Bearer token before streaming the file.

4.4 API Endpoints Used

Endpoint	Method	Client	Purpose
----------	--------	--------	---------

/rest/v1/documents	GET	Mobile + Desktop	List user documents
/rest/v1/documents	POST	Mobile + Desktop	Insert new document row
/rest/v1/documents?id=eq.{id}	PATCH	Mobile + Desktop	Rename document
/rest/v1/documents?id=eq.{id}	DELETE	Mobile + Desktop	Delete document row
/storage/v1/object/scans/{path}	POST	Mobile + Desktop	Upload PDF to bucket
/storage/v1/object/authenticated/scans/{path}	GET	Desktop	Download PDF
/storage/v1/object/scans/{path}	DELETE	Mobile + Desktop	Delete PDF from bucket

5. Desktop–Mobile Communication

Two independent communication channels connect the mobile app to the desktop client. They complement each other: the cloud path is reliable and works anywhere; the Bluetooth path is lower-latency when both devices are nearby.

5.1 Primary Channel: Supabase Cloud Sync

This is the default and most reliable delivery mechanism.

Step	Actor	Action
1	Mobile	User taps "Send to PC" — PDF bytes are in memory
2	Mobile	Upload PDF bytes to Supabase Storage (scans bucket)
3	Mobile	INSERT row in public.documents with metadata
4	Desktop sync thread	Next poll (≤ 12 s) fetches <code>list_user_documents()</code>
5	Desktop sync thread	New <code>doc_id</code> detected; <code>download_pdf()</code> fetches bytes
6	Desktop sync thread	PDF saved to inbox directory; <code>pdf_received</code> signal emitted
7	Desktop MainWindow	Slot adds document to list; PDF is ready to open

De-duplication is handled by the `_received_doc_ids` set in the sync thread and by `document_manager.has_cloud_doc()`: if a document with a given UUID is already in the local inbox, it is skipped without re-downloading.

5.2 Secondary Channel: Bluetooth (SMK2 Protocol)

After the cloud upload completes (or fails), the mobile app attempts a direct peer-to-peer Bluetooth transfer using the **SMK2** binary protocol.

5.2.1 SMK2 Wire Format

Offset	Size	Field
-----	----	-----
0	4 B	Magic header: <code>0x534D4B32</code> ('SMK2')
4	4 B	Payload length in bytes (big-endian <code>uint32</code>)
8	N B	Raw PDF payload (N = payload length)

The receiver (desktop) reads exactly 8 header bytes, validates the magic, reads exactly N payload bytes, and writes the result to the inbox directory. There is no checksum or acknowledgement in the current protocol version.

5.2.2 Transport

- Mobile (Android):** `BluetoothAdapter.getRemoteDevice(mac) → createRfcommSocketToServiceRecord() → connect() → write frame.`
- Desktop (Windows Python):** `socket.socket(AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM) → bind((BDADDR_ANY, 6)) → listen(1) → accept() → read frame.`

- RFCOMM channel: **6**.
- After each received file, the desktop optionally queries Supabase (**find_recent_document_by_file_name**) to link the local file to its cloud doc_id, avoiding a duplicate download on the next sync poll.

5.3 Fallback Strategy

Because Windows Bluetooth RFCOMM can be unreliable (kernel socket errors with message *"read failed, socket might closed or timeout"*), the two channels are designed so that Bluetooth failure is never user-blocking:

- If the cloud upload succeeds but Bluetooth fails → toast: "Sent to cloud — will sync to PC shortly."
- If both succeed → toast: "Sent to PC." (Supabase sync will skip the duplicate.)
- If cloud upload fails → Bluetooth is still attempted; on failure the user sees an error toast.

6. Vision Processing Prototype

A separate experimental module, located under **vision-prototype/**, provides a REST microservice for server-side image processing. It is not integrated into the production flow but serves as a proof-of-concept for future document enhancement features.

6.1 Stack

- **FastAPI** — HTTP framework; auto-generates OpenAPI docs at /docs.
- **OpenCV (cv2)** — Canny edge detection, contour finding, perspective warp (de-skew).
- **Pillow / NumPy** — image I/O and array manipulation.
- **Uvicorn** — ASGI server.

6.2 Endpoints

Endpoint	Method	Input	Output
/process	POST	multipart/form-data image file	Processed JPEG bytes
/health	GET	—	JSON {status: ok}

The **/process** endpoint applies: greyscale conversion → Gaussian blur → Canny edge detection → largest-quadrilateral contour detection → four-point perspective transform. If no document quad is detected the original image is returned unchanged.

7. Key Dependencies

7.1 Mobile (Android / Kotlin)

Library	Version	Purpose
androidx.camera:camera-camera2	1.4.x	CameraX capture backend
androidx.camera:camera-lifecycle	1.4.x	Lifecycle-aware camera
androidx.camera:camera-view	1.4.x	PreviewView Composable
io.github.jan-tennert.supabase:gotrue-kt	3.x	Supabase authentication
io.github.jan-tennert.supabase:storage-kt	3.x	File storage SDK
io.github.jan-tennert.supabase:postgrest-kt	3.x	Database REST client
io.ktor:ktor-client-okhttp	3.x	HTTP engine for Supabase-kt
androidx.compose.*	BOM 2024.x	Jetpack Compose UI toolkit
androidx.navigation:navigation-compose	2.x	Compose navigation (optional)
com.google.android.gms:play-services-auth	21.x	Google Sign-In

7.2 Desktop (Python)

Package	Purpose
PyQt5	GUI framework (widgets, threads, signals)
requests	HTTP client for Supabase REST and Storage APIs
PyMuPDF (fitz)	PDF page rendering for the embedded viewer
Pillow	Image manipulation utilities

7.3 Vision Prototype (Python)

Package	Purpose
fastapi + uvicorn	REST API server
opencv-python (cv2)	Image processing pipeline
numpy	Array operations
Pillow	Image I/O

8. Security Considerations

- **Authentication:** All access is gated on a Google-issued OAuth token validated by Supabase GoTrue. No passwords are stored.
- **Row-Level Security:** Every database operation is subject to RLS policies that compare `auth.uid()` with the row's `user_id`. Cross-user data access is impossible at the database layer.
- **Storage policies:** Equivalent path-prefix policies on the *scans* bucket prevent users from reading or writing each other's files.
- **Token handling:** The desktop client detects JWT expiry locally (via the *exp* claim) and pauses sync rather than making failed requests.
- **Bluetooth:** The RFCOMM channel is unauthenticated at the protocol level; SMK2 does not include a token. This is acceptable because the subsequent Supabase upload/row insertion is still token-protected. The Bluetooth path is treated as an untrusted transport.
- **API keys:** The Supabase anon key is a public key by design (safe to ship in client code); all privilege elevation is handled by RLS, not by keeping the key secret.

9. Repository Structure

```
ScanMeow/
├── mobile-app/                                Android project
│   ├── app/src/main/
│   │   ├── java/com/project/scanmeow/
│   │   │   ├── MainActivity.kt
│   │   │   ├── data/
│   │   │   │   ├── db/                    Room DB or Supabase wrapper
│   │   │   │   ├── model/                Data classes
│   │   │   │   ├── navigation/           NavHost graph
│   │   │   │   ├── ui/
│   │   │   │   │   ├── components/       Reusable Composables
│   │   │   │   │   ├── home/             MainHomeScreen
│   │   │   │   │   ├── screens/          Full-screen Composables
│   │   │   │   │   ├── theme/            Colours, typography
│   │   │   │   │   └── viewmodel/         ViewModels
│   │   └── AndroidManifest.xml
│   └── desktop-app/
│       ├── main.py                          Entry point
│       └── app/
│           ├── main_window.py
│           ├── supabase_client.py
│           ├── supabase_sync.py
│           ├── document_manager.py
│           ├── bluetooth_receiver.py
│           └── pdf_viewer.py
│   └── vision-prototype/
│       ├── main.py                          FastAPI app
│       └── processing.py                    OpenCV pipeline
│   └── supabase/
│       └── migrations/
│           └── 20260327000000_scanmeow_documents.sql
```